

BUILD MANUAL

RaceChrono DIY CAN-Bus アダプタ 製作手順書

ESP32 + SN65HVD230 / Abarth 595 シリーズ4 向け

対象車両	Abarth 595 シリーズ4 (Fiat 500 / 312系)
マイコン	Freenove ESP32-WROOM ボード (FNK0090B)
ファームウェア	katsuhaa / racechrono-diy-esp32 ブランチ <code>abarth595-esp32-wroom</code> (kevenduchesneau 上流 c2290a9 に config.h と本書を追加)
動作モード	受信専用 (Listen Only / TX 無接続)
版数	Rev. 1.7 (配線をシルク表記に統一・ピン配置図と配線図を追加、スニファ関連を削除、config.h をリポジトリと同期)
作成日	2026年9月6日
ビルド確認	arduino-esp32 3.3.11 / NimBLE-Arduino 1.4.3 / EasyLogger 1.1.4 / ESP32 Dev Module (604,492 bytes)

1. 本書の概要と設計方針

本書は、Abarth 595 シリーズ4の OBD-II ポートから CAN バスのデータを読み取り、Bluetooth LE 経由でスマートフォンアプリ「RaceChrono Pro」へ送信する DIY データロガーの製作手順書です。Rev.1.0 の技術レビューと、その後の設計簡素化（保護部品・抵抗・電源スイッチ・外付けコンデンサの削除、TXD 開放）、Fiat 500 系の既知 CAN ID の反映を含みます。Rev.1.7 では配線の記述を各基板に印刷されている名称（シルク表記）に統一し、ピン配置図と配線図を追加しました。変更の経緯は末尾の改訂履歴を参照してください。

設計上の4つの原則

原則	内容
完全受信専用	車両の CAN バスへ一切の信号を送出しない。ファームウェアの Listen Only モードに加え、 トランシーバの送信入力 D を無接続（開放＝レセッシブ固定） にし、ESP32 側から物理的に駆動できないようにする二重の対策を採る。
バス負荷ゼロ	トランシーバ基板上的の終端抵抗を除去し、車両側バスのインピーダンスに影響を与えない。
電源の切り離し	OBD-II の 12V は常時通電のため、使わないときは装置を OBD ポートから物理的に抜く。電源スイッチは設けない。
段階的に確認	机上（USB 給電）→ 机上（12V 給電）→ 車両（USB 給電・12V 線は未接続）→ 車両（12V 運用）の順に、1 段階ずつ動作を確認してから次へ進む。

重要：車両側の安全に関する前提

本装置は走行中の車両の通信バスに接続します。誤配線や設定ミスはパワーステアリング・ABS・エンジン制御に影響を及ぼす可能性があります。**第9章のベンチテストを完了するまで、車両に接続しないでください。**また、公道での初回接続は避け、必ず安全な場所での停車状態から確認を始めてください。

2. 部品表（BOM）

Rev.1.0 から削除した部品：ヒューズ、逆接続保護ダイオード、TVS、10kΩ 抵抗、電源スイッチ、電解コンデンサ×2（経緯は改訂履歴）。残りは 7 品目。

部品	仕様・型番	役割と備考
マイクロコントローラ	Freenove ESP32-WROOM 開発ボード（FNK0090B）	メイン処理・BLE通信。ESP32-WROOM-32E 搭載、デュアルコア240MHz / 4MBフラッシュ / BT 4.2 LE、USB-シリアルは CH340。ピンヘッダは実装済みのため、薄型化のため除去を推奨（第4章）。ピン名はシルクに数字だけで印刷されている（第5章）。
CAN トランシーバ	SN65HVD230 ブレイクアウト基板	CAN 差動信号を 3.3V ロジックへ変換。 基板上の 120Ω 終端抵抗は必ず除去すること （第4章）。
OBD-II コネクタ	SAE J1962 Type A 16ピン オス（金属ラッチ・圧着端子）	車両ポートへの接続。金属ラッチにより強いGでの脱落を防止。付属オープンバレル端子は AWG24～20 対応。

降圧モジュール	MP1584EN 可変DC-DCモジュール	車両12Vを5Vへ降圧。 接続前に必ず5.0Vへ調整 （第6章）。
配線材	AWG24 耐熱シリコンワイヤー	モジュール間結線。IWISS SN-QC-WC で圧着可能、狭い筐体でも取り回し良好。
絶縁・固定材	カプトンテープ、熱収縮チューブ、結束バンド	ショート防止と、筐体・車両ハーネスの物理固定。
筐体	3Dプリント品（タイラップ用スリット付き）	OBDポート周辺は膝が当たるため、可能な限り薄型に設計する。夏季の車内温度を考慮し PLA は避ける。

補足：予備のボードについて

Freenove のボードは2個パックで購入済みのため、1枚はベンチテスト・練習用として温存することを推奨します。ピンヘッダ除去やはんだ付けの失敗リスクに対する保険になります。

3. 対象車両のCANバス仕様

Abarth 595 シリーズ4は Fiat 500（312系）プラットフォームを共有しており、高速側と低速側の2系統のCANバスを持ちます。Fiat 500（312）の C-CAN では、常時放送されるフレームも 29 ビット（拡張）ID で観測されています（例：0x0218A006 車輪速。2010 年式 500 1.2 の公開ログ）。Abarth 595 も同じ形式である可能性が高い（有力）ため、本書の手順は 29 ビット ID を前提に書いています。ファームウェアは全 ID 受け入れなので設定変更は不要です。

バス	速度	OBDコネクタ	主に流れるデータ
CAN-C（高速）	500 kbps	ピン 6 / 14	エンジン制御、ABS/ESC、変速機系。RPM・車速・アクセル開度・ブレーキ圧など
CAN-B（低速）	50 kbps	ピン 1 / 9	ボディ系、メーター、電動パワーステアリング、快適装備。本書では使用しない

本手順書はピン6/14の CAN-C（500kbps）を対象とします。ファームウェアの初期設定が 500kbps のため、ビットレートの変更は不要です（ピン割当は AlfaOBD の Fiat/Alfa 向け資料と一致）。

舵角は CAN-C 側にある見込み（有力）

AiM の「FIAT_ABARTH – CAN STANDARD」プロトコル（Fiat 500 Abarth 2008 年以降）と Racelogic の「Fiat 500 2007-」CAN データベースは、どちらも OBD ピン 6/14（500kbps）を聞くだけで舵角・ブレーキ圧・4 輪速・アクセル開度・回転数などを取得しています。したがって舵角のために CAN-B（ピン 1/9）用の 2 台目を作る必要はない見込みです。CAN-C で見つからなかった場合に限り、2 台目（B-CAN はフォールトトレラント方式の可能性が高く SN65HVD230 での受信は要確認、RaceChrono の 2 台同時接続も要確認）を検討します。

整備済みの CAN ID 一覧は無いが、手がかかりは多い

MX-5 や GT86 のように RaceChrono 用に整備された一覧は Abarth 595 向けには公開されていません。ただし Fiat 500 (312) 系は同じ CAN マトリクスを共有しており、公開情報から次の候補が得られます。第 10 章ではこの候補を最初に確認し、足りないものは OBDLink EX で探します。

CAN ID (29bit)	内容	出典・確度
0x0218A006	4 輪速。8 バイトに 16 ビット × 4 (FL/FR/RL/RR の順は要確認)。停車時は各 0x002C	Fiat 500 1.2 (2010) の公開ログ (P1kachu)。有力
0x0210A006	車速。バイト 4-5 (ビッグエンディアン) ÷ 128 = km/h	Fiat 500L (FiatMon) で復号、Fiat 500 (2010) でも同 ID が車速に連動。有力
0x0618A001	エンジン回転数。バイト 2-3 (ビッグエンディアン) = rpm	Fiat 500L (FiatMon)。Fiat 500 (2010) にも同 ID が存在。五分
0x0810A000	ブレーキ。バイト 2 の上位ニブルが踏力で増加 (0x1x 離し / 0x3x 軽踏み / 0x7x 強踏み)	Fiat 500 (2010) の公開ログ。五分 (圧力のスケールは未確認)
0x0A18A000	状態ビット。バイト 0 ビット 5 = パーキングブレーキ、バイト 2 = ドア / キー状態	Fiat 500 (2010) の公開ログ。五分
0x0628A001	バイト 5 にクラッチ状態ビット	Fiat 500 (2010) の公開ログ。五分
0x0018A001	水温。バイト 3 - 40 = °C	Fiat 500L (FiatMon)。五分
(未特定)	舵角・ブレーキ圧・アクセル開度・ヨーレート・横G	AiM / Racelogic は取得済みだが ID 非公開。OBDLink EX で特定する (第10章)

出典: [github.com/P1kachu/talking-with-cars \(notes/flat-500.txt\)](https://github.com/P1kachu/talking-with-cars/blob/master/notes/flat-500.txt)、github.com/VeryBusyBee/FiatMon (Fiat 500L)、Racelogic Vehicle CAN Database 「Fiat 500 2007-」 (チャンネル一覧のみ公開。ID は .REF ファイル内、可読かは要確認)、AiM InfoTech 「Fiat 500 Abarth (from 2008 onward)」。CANchecked 社は 595 向け製品で解読済みだが非公開。

4. 事前加工 (組み立て前に必ず実施)

4-1. SN65HVD230 基板の120Ω終端抵抗を除去する

市販の SN65HVD230 ブレイクアウト基板の多くには、CANH-CANL 間に 120Ω の終端抵抗が実装されています。車両の CAN バスは既に両端が 120Ω で終端されているため、これを追加すると3本並列となりバスインピーダンスが約 40Ω まで低下し、通信エラーの原因となります。

1. 基板上で CANH と CANL の端子間に配置された抵抗 (表面実装、刻印「121」または「1200」) を特定する。
2. はんだごてで両端を温めながらピンセットで除去する。ジャンパ形式の基板であればカットする。
3. テスターで CANH-CANL 間の抵抗値を測定し、120Ω 付近でないことを確認する。トランシーバ内部の入力抵抗により数十 kΩ 程度が読める、または OL 表示になる。どちらも正常。

4-2. Rs（スロープ制御）ピンの確認

Rs ピンがジャンパで切り替え可能な基板の場合は、高速モード（Rs をグランド側）に設定します。抵抗で固定されている基板はそのまま問題ありません。Rs は主に送信側のスルーレートを決めるピンで、GND 側（高速）または抵抗で GND に落とした状態（スロープ制御）であれば受信には影響しません。

4-3. ESP32 ボードのピンヘッダ除去（任意）

Freenove のボードはピンヘッダがはんだ付け済みで出荷されます（2列 × 20 = 40ピン）。一括で吸い取るとランドが剥がれるため、以下の順序で作業してください。

1. ピンヘッダの黒い樹脂部分をニッパーで破壊し、除去する。
2. 残った金属ピンを1本ずつラジオペンチで掴み、はんだごてで温めながら引き抜く。
3. すべて抜き終わったら、はんだ吸取り線でスルーホールを整える。

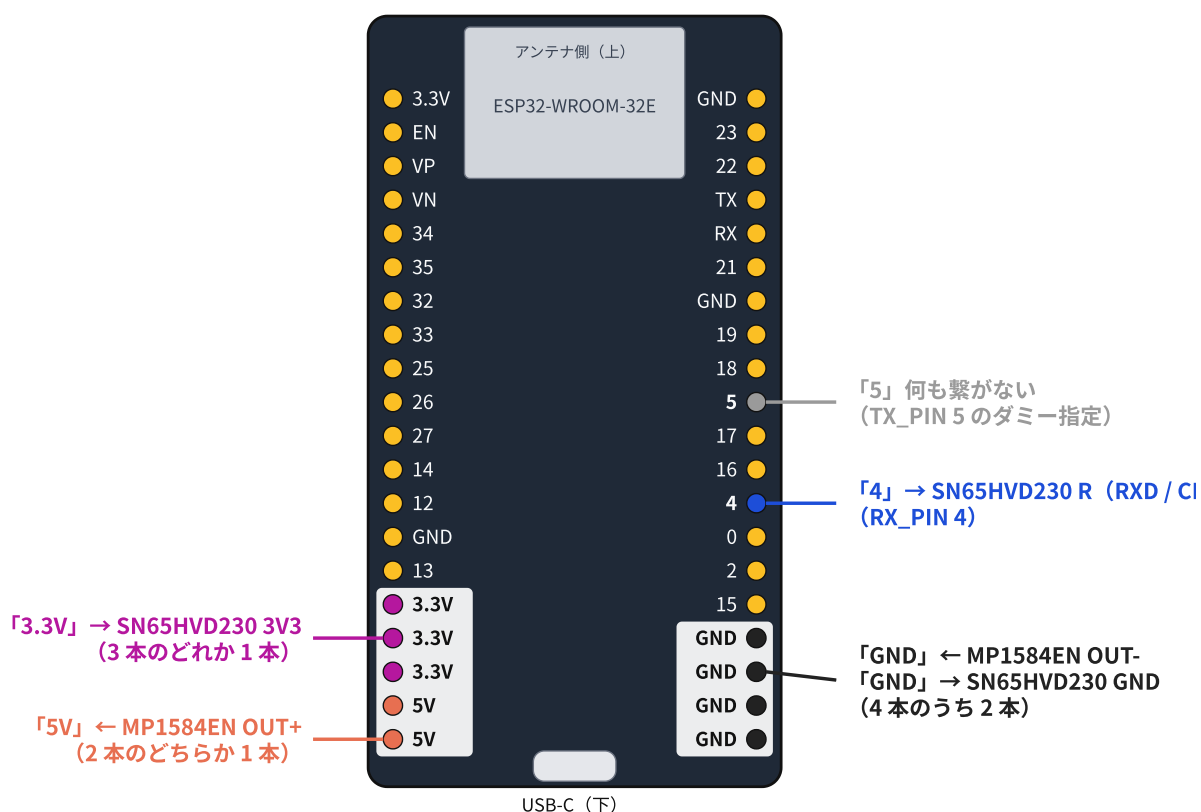
※ 薄型化が不要であれば、この工程は省略しても機能上の問題はありません。ピン名のシルクは各スルーホールの脇に印刷されているので、ヘッダを抜いても読めます。

5. 結線

本章のピン名は、**すべて基板に印刷されている文字（シルク）で表記**します。Freenove ボードでは GPIO 番号が数字だけで印刷されているため、config.h の `RX_PIN 4` はシルクの「4」、`TX_PIN 5` は「5」に対応します。電源ピンのシルクは「3.3V」「5V」「GND」です（「3V3」「VIN」という表記はこのボードにはありません）。

5-1. Freenove ボードで使うピン

アンテナ側（モジュールの金属缶がある側）を上、USB-C コネクタを下にして見ます。使うのは右列の「4」、左列の「3.3V」1本と「5V」1本、右列の「GND」2本の計5本で、「5」には何も繋ぎません。



シルクの文字は各スルーホール脇に印刷されている。ピンヘッダを抜いても残る。

図 5-1 Freenove ESP32-WROOM ボード (FNK0090B) のシルク表記と使用ピン。配置は Freenove 公式のピン配置図 (GitHub リポジトリ Freenove_ESP32_WROOM_Board) に基づく。

シルク	場所	用途
4	右列、上から 13 番目。「16」と「0」の間	CAN 受信入力 (config.h の RX_PIN 4)。SN65HVD230 の R へ
5	右列、上から 10 番目。「18」と「17」の間	何も繋がらない。config.h の TX_PIN 5 はドライバ起動用のダミー指定
3.3V	左列の下側、白枠内の 3 本 (左列の最上段も 3.3V)。どれか 1 本	SN65HVD230 の電源
5V	左列の最下部、白枠内の 2 本。どれか 1 本	MP1584EN の 5.0V 出力を入れる
GND	右列の最下部、白枠内の 4 本 (ほかに右列最上段、「21」の下、左列「12」の下にもある)。2 本使う	MP1584EN の OUT- と、SN65HVD230 の GND

5-2. J1962 プラグ (OBD-II コネクタ) で使うピン

端子を入れる位置は **16・5・6・14 の 4 か所** (GND を 2 本にする場合は 4 も) です。ピン番号の並びは見る向きで左右が反転するので、必ず図 5-2 の③ (端子を差し込む側) で数えるか、プラグ本体の刻印番号を見てください。

① 車両側ソケット（正面から見る）



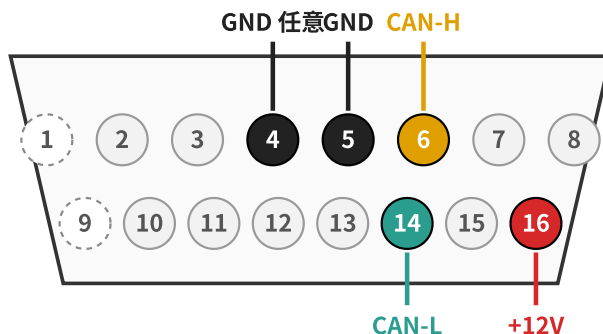
足元のポートをのぞき込んだ向き。上段 1→8、下段 9→16。

② プラグの嵌合面（ピン先端側から見る）



①の鏡像。上段 8→1、下段 16→9。ここで数えると間違えやすい。

③ プラグの配線側（後ろから見る）＝端子を差し込む側



①と同じ並び。本体の刻印番号があればそれを優先。

図 5-2 J1962 のピン配置。①車両側ソケットを正面から、②プラグを嵌合面から、③プラグを配線側（後ろ）から見た図。いずれも広い側を上になっている。

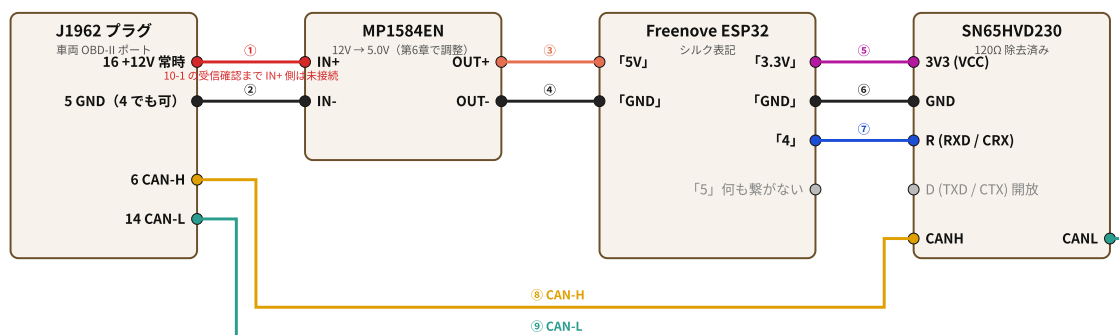
ピン	信号	接続先と備考
16	+12V（バッテリー常時）	MP1584EN の IN+ へ。10-1 の受信確認が終わるまで MP1584EN 側は接続しない（プラグ側の圧着は先にしてよい）
5	GND（信号グラウンド）	MP1584EN の IN- へ。GND はこの 1 本で足りる
4	GND（車体グラウンド）	任意。使う場合はピン 5 の線と一緒に IN- へ
6	CAN-H（CAN-C 高速バス）	SN65HVD230 の CANH へ
14	CAN-L（CAN-C 高速バス）	SN65HVD230 の CANL へ
1・9	CAN-B（低速 50 kbps）	本書では使わない。端子を入れない

※ 付属の端子は AWG24～20 対応で AWG24 には大きめです。被覆を剥いた芯線を折り返して二重にしてから圧着すると抜けにくくなります。圧着後、テスターで 16-5 間および 6-14 間が導通していないことを確認してから車両に挿してください。

5-3. 配線表（1本ずつ）

全9本です。番号は図 5-3 の丸数字に対応します。SN65HVD230 基板と MP1584EN モジュールのピン名は基板の印刷を確認してください（SN65HVD230 の受信出力は R・RXD・CRX のいずれかで印刷されています）。

#	起点	終点	信号	備考
①	プラグ 16	MP1584EN IN+	+12V	10-1 の受信確認まで MP1584EN 側を繋がない
②	プラグ 5（4 でも可）	MP1584EN IN-	GND	
③	MP1584EN OUT+	Freenove 「 5V 」	5.0V	第6章で 5.00V に調整済みのもの
④	MP1584EN OUT-	Freenove 「 GND 」	GND	IN- と OUT- はモジュール内部でつながっている
⑤	Freenove 「 3.3V 」	SN65HVD230 3V3 （VCC）	3.3V	トランシーバ電源。USB 給電のみでも受信動作する（第10章で利用）
⑥	Freenove 「 GND 」	SN65HVD230 GND	GND	
⑦	Freenove 「 4 」	SN65HVD230 R （RXD / CRX）	CAN 受信	受信専用。「5」からは何も出さない
⑧	プラグ 6	SN65HVD230 CANH	CAN-H	6=H、14=L を入れ違えない
⑨	プラグ 14	SN65HVD230 CANL	CAN-L	
—	Freenove 「 5 」	（接続しない）	—	ファームウェア上のダミー指定のみ
—	SN65HVD230 D （TXD / CTX）	（開放。何も接続しない）	—	送信を物理的に無効化する。多くの基板で GND ピンの隣にあるので、ヘッダを残す場合はカプトンテープで絶縁する。より確実にしたい場合はジャンパ線で 3V3 へ結んでもよい（任意）



丸数字は 5-3 の配線表の番号。外付けコンデンサ・スイッチ・抵抗は無し。MP1584EN の IN- と OUT- はモジュール内部でつながっている。

図 5-3 配線図。線の色は 5-3 の表と同じ。

※ 外付けのコンデンサは付けない（MP1584EN モジュールの入出力コンデンサと ESP32 ボード側のコンデンサで足りる見込み・有力）。BLE 接続中に ESP32 がリセットを繰り返す場合だけ、Freenove の「5V」／「GND」間直近に電解 100μF 程度を追加して様子を見る。※ 電源スイッチは設けない。※ プラグ 16 → MP1584EN IN+ の 12V 線は 10-1 の受信確認が終わるまで接続しない（USB 給電で作業する）。12V 線を繋いだ後は、USB を挿すときは装置を OBD から抜く。

「5」を配線せず、TXD を開放にする理由

ファームウェアは TWAI を Listen Only モードで起動します（ソースコードで確認済み）。Espressif の公式ドキュメントでは、Listen Only は ACK もエラーフレームも含めドミナントビットを一切送出しない仕様で、公式エラッタ（ESP32 の TWAI 関連 11 件）にも該当する項目はありません。ただし esp32.com には「Listen mode が純粋な listen ではなく ACK らしきものを出す」という未検証の報告があり、また通常モードでのビットレート不一致など設定ミスがあれば、どの CAN コントローラでもエラーフレームを出します。

本装置は送信機能を一切使わないため、トランシーバの D（TXD）には何も接続せず、ESP32 側の「5」も配線しません。SN65HVD230 は D が High（ $V_{IH} \geq 2V$ ）または開放のとき必ずレセツプになる（データシートのドライバ機能表。D からは約 30μA が流出する仕様で、開放でも内部的に High 側へ引かれる）ため、ソフトウェア側に不具合や設定ミスがあっても物理的にバスを駆動できません。

6. 電源電圧の調整

ESP32 を接続する前に必ず実施

MP1584EN の出荷時出力電圧は不定です。調整前の出力が ESP32 に加わると即座に破損します。

調整は自宅です。MP1584EN の入力範囲は 4.5~28V で、出力電圧の設定は入力電圧に依存しない（フィードバックで一定に保たれる）ため、7~28V の DC 電源なら何でも使え、そこで調整した値がそのまま車両の 12~14.5V でも有効です（有力）。手持ちで使えるもの：9V 乾電池（006P）、電圧を問わず 7~28V の AC アダプタ（ルーター 12V、モニタや古いノート PC の 19~20V、LED 用 12/24V など）、USB 充電器 2 個の直列（各出力は絶縁されているので 5V+5V=10V にできる。USB ケーブルを切って赤=+5V・黒=GND を取り出す）、単3 電池 6 本直列（アルカリ 9V／ニッケル水素 7.2V。無負荷なのでテープで束ねれば十分）。いずれも無ければ車のシガーソケットの 12V で行う（ESP32 は繋がらない）。

1. MP1584EN の IN+／IN- に 9V 乾電池（またはその他 7～28V の DC 電源）を接続する。この時点で ESP32 とトランシーバは接続しない。
2. テスターを OUT+ と OUT- に当て、出力電圧を測定する。
3. 基板上の半固定抵抗（ポテンショメータ）を精密ドライバーでゆっくり回し、5.00V ± 0.05V に調整する。
4. 入力を切り、再投入して電圧が再現することを確認する。
5. 調整完了後、半固定抵抗にマニキュアまたは接着剤を少量塗布して固定する。車両の振動で設定値がずれるのを防ぐため、この工程は省略しないでください。
6. 電源を切ってから、Freenove の「5V」ピンへ接続する（配線③④）。

7. ソフトウェア環境の構築

7-1. Arduino IDE の準備

- Arduino IDE をインストールし、ボードマネージャで esp32 by Espressif Systems (3.x 系) を導入する。本書は 3.3.11 でビルドを確認済み。作者の開発環境も 3.x 系。
- macOS で書き込みポートが表示されない場合は、CH340 ドライバ（Freenove の GitHub リポジトリに同梱）を導入する。

7-2. 必要ライブラリ

ライブラリ	導入方法	備考
NimBLE-Arduino	ライブラリマネージャでバージョン 1.4.3 を明示的に選択	BLE スタック。既定でインストールされる 2.x（現行 2.5.1）では arduino-RaceChrono がビルドできない（setScanResponse が廃止、onWrite の引数が変更）。1.4.3 は 1.x 系の最終版で、esp32 3.x 系に対応。
EasyLogger	ライブラリマネージャ（1.1.4 で確認）	シリアルログ出力
arduino-RaceChrono	手動インストール	ライブラリマネージャには登録されていない。下のコマンドで ~/Documents/Arduino/libraries に配置する（macOS の Arduino IDE の既定スケッチブック）。

```
cd ~/Documents/Arduino/libraries
git clone https://github.com/timurrrrr/arduino-RaceChrono.git arduino-RaceChrono
```

7-3. ソースコードの取得

1. 自分のフォーク katsuhaa/racechrono-diy-esp32 のブランチ abarth595-esp32-wroom を使う。上流 kevenduchesneau/racechrono-diy-esp32（コミット c2290a9）に、Abarth 用の config.h と本書（docs/）を追加したもの。手元に無い場合は次のコマンドで取得する。

```
cd ~/Documents/workspace/ODB2CloneForRaceChrono
git clone -b abarth595-esp32-wroom https://github.com/katsuhaa/racechrono-diy-esp32.git
```

2. Arduino IDE で `racechrono-diy-esp32/racechrono-diy-esp32.ino` を開く。同じフォルダの `config.h` がタブとして表示されればよい。ブランチに `config.h` が無い場合は `config.h.example` をコピーして `config.h` とし、7-4 の内容にする（このファイルが無いとビルドできない）。
3. 設定値は 7-4 のとおり。変更が必要になるのは `getUpdateRateHz()` の候補 ID（第10章で確認後にコメントを外す）だけ。

7-4. config.h（全文・リポジトリと同一）

```
#ifndef CONFIG_H
#define CONFIG_H

// Configuration for the Abarth 595 Series 4 (Fiat 500 / 312 platform),
// classic ESP32-WROOM-32E board (Freenove FNK0090B), receive-only build.
// Abarth 595 シリーズ4 / 無印 ESP32-WROOM (Freenove FNK0090B) 向け・受信専用の設定。

// Bluetooth device name shown in the RaceChrono app
// RaceChrono アプリに表示される Bluetooth デバイス名
#define DEVICE_NAME "RaceChrono DIY ESP32"

// Serial logging level, select one from the following list :
// LOG_LEVEL_EMERGENCY
// LOG_LEVEL_ALERT
// LOG_LEVEL_CRITICAL
// LOG_LEVEL_ERROR
// LOG_LEVEL_WARNING
// LOG_LEVEL_NOTICE
// LOG_LEVEL_INFO
// LOG_LEVEL_DEBUG
// DEBUG prints the TWAI / ring buffer / BLE frame counts every 10 seconds,
// which is the only on-device proof that CAN frames are being received.
// DEBUG にすると 10 秒ごとに TWAI・リングバッファ・BLE の件数が出る（CAN 受信の唯一の確認手段）。
#define LOG_LEVEL LOG_LEVEL_DEBUG

// GPIO pins connected to the CAN transceiver (SN65HVD230)
// RX_PIN : wired to the transceiver R (RXD / CRX) output
// TX_PIN : INTENTIONALLY NOT WIRED. The TWAI driver needs a valid GPIO number,
//          but nothing is connected to GPIO 5 and the transceiver D (TXD / CTX)
//          input is left open (open = recessive), so the bus can never be driven.
// RX_PIN : トランシーバの R (RXD / CRX) へ配線
// TX_PIN : 意図的に未配線（ドライバ起動用のダミー指定）。トランシーバの D (TXD / CTX) は開放＝レセシブ固定。
#define RX_PIN 4
#define TX_PIN 5

// Serial baud rate
#define SERIAL_BAUD_RATE 115200

// TWAI RX buffer size
// Reduce to 64 if "Failed to create ring buffer." appears at boot
// 起動時に "Failed to create ring buffer." が出る場合は 64 に減らす
#define TWAI_RX_QUEUE_LENGTH 128

// Bluetooth 5.0 support, 1 to enable, 0 to disable
// The classic ESP32 is BT 4.2 and has no extended advertising: keep this commented out
// 無印 ESP32 は BT 4.2 のため有効化不可。コメントアウトのままとする
// #define CONFIG_BT_NIMBLE_EXT_ADV 1

// Limit the rate of messages sent to RaceChrono to x CAN messages each second
// Can be further refined per PID through the list below
```

```
// Start low and raise only the channels that matter in getUpdateRateHz()
// まず低めにし、必要なチャンネルだけ getUpdateRateHz() で引き上げる
#define DEFAULT_UPDATE_RATE_HZ 2

// An optional list of PIDs and their associated rate limit.
// Limits the update rate of the specified PID to the returned value.
// Fiat 500 (312) family broadcast frames use 29-bit IDs. The candidates below
// come from public Fiat 500 / 500L logs (build manual, chapter 3) and are NOT
// yet confirmed on the Abarth 595: uncomment a case once the ID is verified.
// Fiat 500 (312) 系の放送フレームは 29 ビット ID。候補は手順書 3 章の公開情報で、
// Abarth 595 では未確認。実車で確認できたらコメントを外す。
uint8_t getUpdateRateHz(uint32_t can_id)
{
    switch (can_id)
    {

        // case 0x0218A006: // Wheel speeds, 4 x 16-bit / 4 輪速
        //     return 20;
        // case 0x0210A006: // Vehicle speed, bytes 4-5 big-endian / 128 = km/h / 車速
        //     return 10;
        // case 0x0618A001: // Engine RPM, bytes 2-3 big-endian / 回転数
        //     return 10;
        // case 0x0810A000: // Brake, byte 2 upper nibble / ブレーキ
        //     return 20;

        default:
            return DEFAULT_UPDATE_RATE_HZ;
    }
}

#endif // CONFIG_H
```

7-5. Arduino IDE のボード設定

リポジトリの README は ESP32-S3 を前提としているため、無印 ESP32 では以下のように読み替えます。「ESP32 Dev Module」の既定値と異なるのは Upload Speed のみです。

README の記載	無印ESP32での設定
Board	ESP32 Dev Module
USB CDC On Boot	項目なし → 無視
Upload Mode: USB-OTG CDC (TinyUSB)	項目なし → 無視 (通常の UART 書き込み)
USB Mode: Hardware CDC and JTAG	項目なし → 無視
Events Run On: Core 1	Core 1 (既定値。作者は必須と明記)
Arduino Runs On: Core 1	Core 1 (既定値。作者は必須と明記)
Partition Scheme: Default	Default 4MB with spiiffs (1.2MB APP) のまま。ビルド結果 604,492 bytes (46%) で収まる
Upload Speed	115200 (921600 でも書けるが、失敗時は 115200 に落とす)

7-6. 書き込みと起動確認

1. 装置を車両 (OBD) に繋いでいない状態で、USB ケーブルで PC と接続する。

2. ポートを選択し、スケッチを書き込む。
3. シリアルモニタ (115200bps) を開き、起動ログに `Driver started.` と `BLE started.` が出ることを確認する。LOG_LEVEL が DEBUG のため、以後 10 秒ごとに `0 messages received from TWAI in the last 10 seconds.` が出る。机上では 0 で正常。

USB 給電と車両 12V の同時接続について

Freenove ボードは回路図が公開されておらず、USB の 5V と「5V」ピンの間に逆流防止ダイオードがあるかは不明です。本書では「USB 給電と 12V 給電を同時にしない」を原則とします。書き込み・シリアル確認はすべて USB 給電のみで行い、車内でのシリアル作業 (10-1) はプラグ 16 → MP1584EN の 12V 線を繋ぐ前に済ませます (トランシーバは Freenove の「3.3V」から給電されるため、USB 給電だけで CAN 受信も動作します)。12V 線を繋いだ後に USB を使うときは、装置を OBD から抜きます。同時給電が必要な場合は、USB を外した状態でテストのダイオードモードで USB コネクタの VBUS と「5V」ピン間を測り、ダイオード (順方向 0.2~0.7V 程度、逆方向は OL) が確認できた場合に限り可とします。

8. 性能に関する期待値

- 作者の README では、ESP32-S3 で 120Hz 級 (合計では約 200Hz、Google Pixel 7 で計測) に達し、シングルコアの ESP32-C3 でも同等、BLE 5.0 の拡張アダプタイズを有効にしても差が無かったと記載されています。ボトルネックは BLE リンクの転送速度です。
- したがって BT 4.2 の無印 ESP32 でも S3 と同程度が見込めます (有力)。実効レートはスマートフォン側 (iOS は接続間隔の下限が長め) に左右されるため、実測して決めます。
- 設定方針: `DEFAULT_UPDATE_RATE_HZ` は 2 から始め、本当に必要なチャンネル (舵角・ブレーキ・アクセル・車速/回転) のみ `getUpdateRateHz()` で 20~25Hz に引き上げる。すべての PID を高レートで通そうとすると BLE 側で詰まり全体のレートが落ちます。
- 実測方法: LOG_LEVEL は既定で DEBUG なので、10 秒ごとに「messages received from TWAI」「messages sent to ring buffer」「messages sent to RaceChronoBle」が出ます。ring buffer 行と RaceChronoBle 行の差が広がる、または `Receive ring buffer overflow` が出るなら BLE が飽和しています。レートを下げてください。

9. ベンチテスト (車両接続前の必須確認)

以下をすべて満たすまで、車両には接続しないでください。CAN の受信そのものは本 BOM では机上で確認できない (送信側が無い) ため、第10章の初回接続で確認します。

9-1. 通電前 (テスター)

- ☐ SN65HVD230 基板の CANH-CANL 間が 120Ω 付近でない (120Ω 除去済み。数十 kΩ 以上または OL)
- ☐ MP1584EN の出力が 5.00V ± 0.05V で安定しており、半固定抵抗が固定されている
- ☐ プラグ 16 → MP1584EN IN+、プラグ 5 (/4) → IN- の極性が正しい (逆接続保護は無い)
- ☐ Freenove の「5」から配線が出ていない。「4」の線が SN65HVD230 の R (RXD / CRX) に入っている
- ☐ トランシーバの D (TXD / CTX) ピンに何も接続されておらず、GND や隣のピンと接触していない (テスターで D-GND 間が導通しない)

- 12V ラインと GND ラインの間、および CANH-CANL 間にショートが無い
- すべての配線に熱収縮チューブまたはカプトンテープで絶縁処理がされている

9-2. USB 給電（12V 未接続）

- シリアルに起動ログが出る（Driver started. / BLE started.、10 秒ごとに 0 messages received from TWAI ...）
- スマートフォンで「RaceChrono DIY ESP32」が見える（iOS は設定 > Bluetooth の一覧、または BLE スキャナアプリ）
- RaceChrono Pro からデバイスを追加し、接続が確立する（シリアルに RaceChrono connected.）

9-3. 外部給電（USB を外し、9V 乾電池を MP1584EN の IN+ / IN- に当てる）

- 給電すると ESP32 が起動し、スマートフォンから BLE が見える・接続できる（USB は外しておく。プラグ 16 の線はまだ IN+ に繋がなくてよい）
- MP1584EN・ESP32 に発熱・異臭が無い。テスターで Freenove の「5V」ピンが 5.0V 付近
- 給電を外すと BLE が消える

10. 車両接続とCAN ID特定作業

10-1. 初回接続（受信確認）

1. プラグ 16 → MP1584EN IN+ の 12V 線はまだ接続しない（USB 給電で作業する）。OBD-II ポート（運転席足元・低い位置）に接続する。ケーブルは上方向へ逃がし、結束バンドで既存ハーネスへ固定する。
2. ノート PC の USB で給電し、シリアルモニタ（115200bps）を開く。
3. イグニッションを ON にする（エンジン始動でもよい）。
4. 10 秒ごとに N messages received from TWAI in the last 10 seconds. が出て、N が 0 より大きければ受信成功。0 のままなら第11章へ。
5. 装置を OBD から抜き、プラグ 16 の線を MP1584EN IN+ に接続して、USB なしで OBD に挿し直し、RaceChrono Pro から接続できることを確認する。

ここでフレームが流れてくれば、本プロジェクトは9割成功です。

10-2. CAN ID の特定（OBDLink EX を使う）

RaceChrono には未知の CAN ID を一覧化したり、全フレームを保存したりする機能はありません（プロトコル上の「allow all」はアプリが PID 編集画面を開いたときに内部的に送るコマンドで、ユーザー操作ではありません）。また上流ファームウェアの DEBUG ログは受信件数しか出しません。そこで、手持ちの OBDLink EX（MultiECUScan 用）を STN コマンドで生 CAN のモニタにして ID を確認します。DIY 機とは独立した作業なので、組み立て前でも行えます。MultiECUScan 自体は診断要求（0x18DA10F1 など）を投げて応答を読む方式で常時放送フレームの ID は表示できませんが、Live Data で舵角などの基準値を読むのに使えます（OBDLink は同時には使えないので、静止状態で交互に照合する）。

Mac のターミナルでポート名を確認し、接続します。

```
ls /dev/tty.usbserial*
screen /dev/tty.usbserial-XXXX 115200
```

接続後、1 行ずつ送ります (Enter で確定)。

```
ATZ
ATE0
ATH1
ATS1
ATCAFE
STP 32
STMA
```

STP 32 は OBDLink FRPM のプロトコル表で「ISO 11898, 29-bit Tx, 500kbps, var DLC」(生 CAN・500kbps)。CAN プリセットは 11 ビット/29 ビットの両方を受信します。STMA で全フレームが流れ続け、29 ビット ID は 02 18 A0 06 00 2C 00 2C ... のように先頭 4 バイトが ID、続きがデータです (11 ビット ID は 7E8 03 41 ... のように 3 桁)。止めるときは何かキーを押す。screen の終了は Ctrl-A、K、y。

1. **既知候補の確認**：500kbps のバスは毎秒数千フレームなので、全量表示では目で追えません。候補 ID だけ通すパスフィルタを入れてから STM で見ます (例は車輪速。フィルタは複数追加でき、STFAC で全消去)。停車中は各輪 00 2C、車を少し動かすと値が変わる、0x0618A001 のバイト 2-3 が回転数、0x0810A000 のバイト 2 上位ニブルがブレーキ踏力で変わる、といった対応を確認します。

```
STFPA 0218A006, 1FFFFFFF
STM
```

2. **未知の ID (舵角・アクセル開度・ブレーキ圧)**：「何も操作しない 20 秒」と「1 つの操作だけを繰り返す 20 秒」の 2 回ログを取り、後者でだけ値が動いた ID とバイトを探します。ログ取得と比較は付録 A のスクリプトで行えます (任意。screen のログでも可)。舵角はロック to ロックで単調に増減する 16 ビット値 (符号付きが多い)、ブレーキ圧は連続値、アクセル開度は回転数と同時に動くので、開度だけを見たい場合はエンジン停止・イグニッション ON でも試します。
3. 候補が決まったら RaceChrono の CAN-Bus チャンネル編集で PID と計算式 (bytesToUIntLe 等) を入力し、実車操作に値が追従するか確認します (10-3)。特定した ID は getUpdateRateHz() に追記して優先度を上げます。

※ 115200bps では取りこぼして BUFFER FULL が出ますが、変化する ID を見つける目的なら実害はありません。全量が必要なら STBR 2000000 で 2Mbps に上げます (screen は 2Mbps 非対応のため付録 A のスクリプトの baud を変更して使う)。診断機自身の要求/応答 (18 DA ...) は無視します。

補足：診断フレームとの見分け方

診断 (KWP/UDS) のフレームは 0x18DAxxF1/0x18DBxxF1 (要求) と 0x18DAF1xx (応答) の形で、診断機を繋いだときだけ流れます。常時放送のフレームは 0x0XXXA0YY の形の 29 ビット ID です。ファームウェアは全 ID 受け入れ設定なので両方受信します。RaceChrono の PID 欄には 29 ビット ID をそのまま入力します (表記形式は実機で確認)。

10-3. RaceChrono Pro 側の設定

1. 公式チュートリアルを経路は「Settings → Add other device → Add other device → RaceChrono DIY」(日本語 UI では「設定 → その他のデバイスを追加 → ...」)。BLE スキャンの一覧から RaceChrono DIY ESP32 を選ぶ。

2. デバイス追加後、車両プロファイルに「CAN-Bus チャンネル」の項目が現れる。特定した CAN ID と計算式を 1 チャンネルずつ登録する。

※ DIY BLE デバイスの追加と CAN-Bus チャンネル機能には RaceChrono Pro（有料版）が必要です。画面名や並びはアプリのバージョンで変わるため、実機で確認したうえで本書を更新してください。

11. トラブルシューティング

症状	確認事項
ビルドエラー： <code>'class NimBLEAdvertising' has no member named 'setScanResponse'</code>	NimBLE-Arduino が 2.x になっている。ライブラリマネージャで 1.4.3 を選んで入れ直す（第7章）。
ビルドエラー： <code>RX_PIN</code> 等が未定義	<code>config.h</code> がスケッチと同じフォルダにあるか確認。マクロ名は <code>CAN_RX_PIN</code> ではなく <code>RX_PIN</code> 。
ビルドエラー： <code>RaceChrono.h</code> が見つからない	arduino-RaceChrono ライブラリを <code>libraries</code> フォルダへ手動配置したか確認（7-2）。
ビルドエラー： <code>driver/twai.h</code> や <code>esp_mac.h</code> が見つからない	ESP32 ボードパッケージが古い、または壊れている。esp32 3.x 系を入れ直す。
起動時に <code>Failed to create ring buffer.</code>	RAM 不足。 <code>.ino</code> 内の <code>xRingbufferCreate(65536, ...)</code> を 16384 に、 <code>TWAI_RX_QUEUE_LENGTH</code> を 64 に減らす。
起動時に <code>Driver failed to start.</code>	<code>RX_PIN</code> / <code>TX_PIN</code> の GPIO 番号を確認。 <code>TX_PIN</code> は未配線でも有効な GPIO 番号を指定する必要がある。
BLE で見えるが TWAI の受信数が 0 のまま	イグニッション ON か。Freenove「4」と SN65HVD230 R（RXD / CRX）の結線、「3.3V」からのトランシーバ給電、CANH/CANL の入れ違い（プラグ 6=H、14=L）、終端抵抗除去時のパターン断線を確認。
受信数は増えるが RaceChrono に値が出ない	PID・計算式の誤り。DEBUG ログの <code>Allow PID ...</code> でアプリからの要求が届いているか確認。
フレームが一切受信できない（上記を確認済み）	ビットレートを確認。500kbps 以外の可能性がある場合は、車両から切り離れた上で <code>.ino</code> の <code>TWAI_TIMING_CONFIG_500KBITS()</code> を変更する。
データレートが遅い・途切れる	BLE の帯域上限。第8章の指標で飽和を確認し、 <code>DEFAULT_UPDATE_RATE_HZ</code> を下げ、必要な PID のみ個別に引き上げる。
ESP32 が起動しない・発熱する	直ちに電源を切る。MP1584EN の出力電圧を再測定する。
放置後にバッテリーが上がる	装置を挿したまま放置した。OBD-II ピン 16 は常時通電なので、使わないときは抜く。

12. 運用上の注意

- 使わないときは装置を OBD ポートから抜く。本装置は挿している間、12V 側で数十 mA を消費し続けます（目安として 1 日あたり 1Ah 前後）。挿したまま放置すると 1~2 週間でバッテリーに影響が出ると見込んでください。
- 夏季の車内は高温になります。長期駐車時は本体を取り外すことを推奨します。
- OBD ポート周辺は乗降時に膝が当たる位置です。本体とケーブルが運転操作を妨げないように固定してください。
- 走行中に操作が必要な状態にしないでください。設定変更は必ず停車時に行ってください。
- 本装置は受信専用ですが、車両の電気系統に接続する改造行為です。実施は自己責任で行ってください。
- 将来の改善案：CAN の無音（車両スリープ）を検出して deep sleep に入り、「4」の信号で復帰する処理を入れると、挿しっぱなしでも消費を抑えられます。

参考リンク

- ファームウェア（自分のフォーク）：github.com/katsuhaa/racechrono-diy-esp32 / 上流：github.com/kevenduchesneau/racechrono-diy-esp32
- ライブラリ：github.com/timurrrr/arduino-RaceChrono
- BLE DIY プロトコル仕様：github.com/aollin/racechrono-ble-diy-device
- CAN ID 事例集（他車種の参考）：github.com/timurrrr/RaceChronoDiyBleDevice
- RaceChrono DIY チュートリアル：racechrono.com/article/2572 / DIY フォーラム：racechrono.com/forum/categories/diy-builds
- NimBLE-Arduino 1.x→2.x 移行ガイド：github.com/h2zero/NimBLE-Arduino/blob/master/docs/1.x_to2.x_migration_guide.md
- ESP-IDF TWAI ドキュメント / ESP32 エラッタ：docs.espressif.com
- Freenove ESP32-WROOM ボード（ピン配置図・CH340 ドライバ）：github.com/Freenove/Freenove_ESP32_WROOM_Board
- Fiat/Alfa の OBD ピン割当（C-CAN 6/14、B-CAN 1/9）：alfaobd.com/androidhelp.html
- OBDLink Family Reference and Programming Manual（STP / STM / STMA / STFPA / STBR）：scantool.net
- Fiat 500（2010）CAN 解析ノート：github.com/P1kachu/talking-with-cars/blob/master/notes/fiat-500.txt
- Fiat 500L CAN 復号（FiatMon）：github.com/VeryBusyBee/FiatMon
- Racelogic Vehicle CAN Database「Fiat 500 2007-」：racelogic.co.uk（PDF） / vboxmotorsport.co.uk（.REF）
- AiM InfoTech「Fiat 500 Abarth (from 2008 onward)」：aim-sportline.com/download/ecu/stock/fiat/FIAT500Abarth_106_eng.pdf

付録 A. OBDLink EX 用 CAN ログ取得・比較スクリプト（任意）

10-2 の「未知の ID」探しを補助する Python スクリプトです。ファームウェアの変更はありません。任意のフォルダに `obdlink_canlog.py` として保存し、`pyserial` を入れて使います。`capture` は OBDLink を生 CAN 500kbps に設定して指定秒数のフレームをファイルに保存し、`compare` は「ログ A では静止していたのにログ B で変化したバイト」を持つ ID と、B にしか現れない ID を表示します。常に変化しているカウンタやチェックサムバイトは A でも変化するの除外されます。ロジックは合成データで検証済み、実機での動作は未検証です。

```
python3 -m pip install pyserial
python3 obdlink_canlog.py capture /dev/tty.usbserial-XXXX 20 rest.log
```

```
python3 obdlink_canlog.py capture /dev/tty.usbserial-XXXX 20 steer.log
python3 obdlink_canlog.py compare rest.log steer.log
```

1 回目は何も触らずに、2 回目はステアリングをロック to ロックでゆっくり往復させながら取得します。出力例：

0628A001 2000 frames changed only in B -> byte0:55 values byte1:256 values のように、操作中にだけ動いた ID とバイト位置が出ます。ブレーキ・アクセルも同様に 1 操作ずつ行います。

```
#!/usr/bin/env python3
# obdlink_canlog.py - OBDLink EX (STN) raw CAN logger and log comparator
# for finding CAN IDs on the Abarth 595 (build manual, chapter 10).
#
# python3 obdlink_canlog.py capture /dev/tty.usbserial-XXXX 20 rest.log
# python3 obdlink_canlog.py capture /dev/tty.usbserial-XXXX 20 steer.log
# python3 obdlink_canlog.py compare rest.log steer.log
#
# capture: sets the OBDLink to raw CAN 500 kbps (STP 32), monitors all frames
#          (STMA) for N seconds and writes the lines to a file. Needs pyserial.
# compare: lists the IDs whose bytes were static in log A but changed in log B,
#          and the IDs that appear only in log B.

import sys
import time

INIT = ["ATZ", "ATE0", "ATL0", "ATS1", "ATH1", "ATCAF0", "STP 32"]
HEX = set("0123456789ABCDEF")

def capture(port, seconds, out):
    import serial # pip3 install pyserial
    ser = serial.Serial(port, 115200, timeout=0.2)
    for c in INIT:
        ser.write((c + "\r").encode())
        time.sleep(1.2 if c == "ATZ" else 0.3)
        print(c, "->", ser.read(ser.in_waiting or 1).decode(errors="replace").strip())
    ser.reset_input_buffer()
    ser.write(b"STMA\r")
    t0 = time.time()
    n = 0
    try:
        with open(out, "w") as f:
            while time.time() - t0 < seconds:
                line = ser.readline().decode(errors="replace").strip()
                if line:
                    f.write(line + "\n")
                    n += 1
    except KeyboardInterrupt:
        pass
    ser.write(b"\r") # any character stops the monitor
    time.sleep(0.5)
    ser.close()
    print(n, "lines ->", out)

def parse(path):
    """Returns {can_id: {"count": n, "vals": [set of byte values] * 8}}."""
    ids = {}
    for line in open(path, errors="replace"):
        parts = line.split()
        if not parts or not all(set(p) <= HEX for p in parts):
            continue # prompt, BUFFER FULL, etc.
```

```

        if len(parts[0]) == 3:                                # 11-bit ID: "7E8 03 41 ..."
            cid, data = parts[0], parts[1:]
        elif len(parts) >= 4 and all(len(p) == 2 for p in parts[:4]):
            cid, data = "".join(parts[:4]), parts[4:] # 29-bit ID: "02 18 A0 06 ..."
        else:
            continue
        e = ids.setdefault(cid, {"count": 0, "vals": [set() for _ in range(8)]})
        e["count"] += 1
        for i, b in enumerate(data[:8]):
            if len(b) == 2:
                e["vals"][i].add(b)
    return ids

def compare(a, b):
    A, B = parse(a), parse(b)
    print("%s: %d IDs / %s: %d IDs" % (a, len(A), b, len(B)))
    for cid in sorted(B):
        eb = B[cid]
        if cid not in A:
            print("NEW %-8s %5d frames" % (cid, eb["count"]))
            continue
        ea = A[cid]
        hits = [i for i in range(8) if len(ea["vals"][i]) <= 1 and len(eb["vals"][i]) > 1]
        if hits:
            detail = " ".join("byte%d:%d values" % (i, len(eb["vals"][i])) for i in hits)
            print("    %-8s %5d frames changed only in B -> %s" % (cid, eb["count"], detail))

if __name__ == "__main__":
    if len(sys.argv) >= 5 and sys.argv[1] == "capture":
        capture(sys.argv[2], float(sys.argv[3]), sys.argv[4])
    elif len(sys.argv) == 4 and sys.argv[1] == "compare":
        compare(sys.argv[2], sys.argv[3])
    else:
        print("usage: obdlink_canlog.py capture PORT SECONDS OUTFILE")
        print("    obdlink_canlog.py compare LOG_A LOG_B")

```

改訂履歴

版	日付	内容
Rev. 1.0	2026-09-04	初版

Rev. 1.1	2026-09-05	7-2：NimBLE-Arduino を 1.4.3 に固定（2.x では arduino-RaceChrono がビルド不能。実際のエラーを 11 章に記載）。arduino-RaceChrono の導入コマンドを追記。7-1：「esp_mac.h のため 2.x 系ではビルド不可」を削除（esp_mac.h は 2.x 系にも存在）。3.3.11 でのビルド確認結果を追記。7-6／9 章／10-1：USB 給電と車両 12V の扱いを整理。書き込み・シリアル確認は「電源スイッチ OFF・USB 給電のみ」に統一。ベンチテストを通電前／USB／12V の 3 段階に再構成。10-2：RaceChrono 側で全 PID をログする手順を削除し、付録 A のスニファによる手順に置換。10-3：公式チュートリアルを経路に修正。裏付けの無い「デバイス種別の設定」を削除し、実機確認を明記。5 章コラム：「既知のエラッタ」を公式資料の記述に合わせて修正（設計は変更なし）。8 章：無印 ESP32 の性能期待値を「S3 と同程度・実測で決める」に修正し、飽和の判定方法を追記。4-1／9 章：終端抵抗除去の判定を「OL」から「120Ω 付近でない」に修正。4-3：40 ピンに修正。3 章：2 台構成の未確認事項（B-CAN の物理層、複数 DIY デバイス）を追記。12 章：バッテリー消費の目安を修正。部品表は変更なし。
Rev. 1.2	2026-09-05	部品表からヒューズ・逆接続保護ダイオード（1N5819）・TVS（SMBJ26A）を削除。5-1 の電源系統と 9-1 のチェックリストを追従。それ以外の章は変更なし。
Rev. 1.3	2026-09-05	10kΩ 抵抗を削除し、トランシーバの D（TXD）は 3V3 へ直結に変更（データシート上 D は High または開放でレセツプ。抵抗は不要）。5-2・5 章コラム・9-1・config.h のコメントを追従。
Rev. 1.4	2026-09-05	D（TXD）を「開放（無接続）」に変更。配線工程を減らして作業を確実にするため。D ピンの絶縁と D-GND 非導通のチェックを追加。3V3 へのジャンパ線は任意扱い。
Rev. 1.5	2026-09-05	3 章：Fiat 500（312）系の公開情報から既知 CAN ID 候補表を追加（車輪速・車速・回転数・ブレーキ・状態ビット）。放送フレームは 29 ビット ID である旨に修正（旧版の「11 ビット」は誤り）。AiM／Racelogic の資料により舵角は CAN-C 側にある見込みと記載し、2 台構成は予備案に格下げ。10-2 に「既知候補の確認」を追加、診断フレームの補足を修正。付録 A：スニファの ID 表示を 29 ビットは 8 桁に変更。
Rev. 1.6	2026-09-05	電源スイッチと外付け電解コンデンサ（100μF×2）を部品表と結線から削除（スイッチは使わないときに OBD から抜けば足りる。コンデンサは DC-DC モジュール搭載分で代替、リセットが出る場合のみ 5V 側に追加）。USB と 12V の同時給電を避ける手順を「CAN ID 特定が終わるまで 12V 線を繋がない」に変更し、7-6・9 章・10-1・12 章を追従。6 章：出力調整は 7～28V の任意の DC 電源（9V 乾電池など）で自宅のできる旨と電源の選択肢を追記。10-2：OBDLink EX を STN コマンドでスニファとして使う代替手段を追加。

Rev. 1.7	2026-09-06	5 章：配線の記述を各基板のシルク表記に統一（Freenove は「4」「5」「3.3V」「5V」「GND」）。Freenove ボードのピン配置図（図 5-1）・J1962 ピン配置図（図 5-2）・配線図（図 5-3）を追加。配線表を 1 本ずつの 9 本に書き直し、旧版で抜けていた MP1584EN OUT- → GND を追記。GND はプラグ 5 の 1 本で足りる旨と、端子を入れるプラグ位置を明記。コラム見出しを Rev.1.4 の「開放」に合わせて修正。7-3/7-4：ソースを自分のフォークのブランチ abarth595-esp32-wroom に変更し、config.h をリポジトリの内容と同期（LOG_LEVEL の既定を DEBUG に変更、スニファ定義を削除）。7-5：ビルドサイズを 604,492 bytes に更新。10-1：DEBUG への切替手順を削除し、12V 線の接続を受信確認の直後に変更。10-2：ESP32 ファームウェアのスニファ手順を削除し、OBDLink EX（STN コマンド）による確認を本線に変更。付録 A をファームウェア差分から PC 側のログ取得・比較スクリプト（任意）に差し替え。9-3：9V 電池は MP1584EN の IN+／IN- に当てる表現に修正。3 章・11 章のスニファへの言及を削除。部品表は変更なし。
----------	------------	--